System Documentation

Database schema structure:

ישויות:

- **movies**: טבלת הסרטים, המכילה רשומה לכל סרט השמור במאגר של TMDB, וכוללת פרטים כמו המזהה שלו, השם שלו, תקציר, סלוגן, שנת יציאה, משך, מדד פופולריות (עפ"י traffic, trending), סטטיסטיקות דירוג TMDB (כמה מבקרים דירגו אותו ומה הדירוג הממוצע שהתקבל בסקאלה של 0-10), תקציב, ועוד. זאת הטבלה המרכזית שאיתה אנחנו עובדים באפליקציה.
  - **מפתח ראשי**: tmdb_id – מזהה הסרט.
  - **דוגמה לרשומה**:

	tmdb_id	imdb_id	title	original_title	overview
1	702	tt0044081	A Streetcar Named Desire	A Streetcar Named Desire	Disturbed Blanche DuBois moves in with her sister in New Orleans and is tormented by her brutish brother-in-law while her reality crumbles around her.

tagline	release_date	release_year	original_language	runtime	status	budget	revenue	popularity	vote_average	vote_count	adult
...Blanche, who wanted so much to stay a lady...	1951-09-18	1951	en	125	Released	1800000	8000000	7.301719	7.6	291	0

- **genres**: טבלת הז'אנרים, הכוללת את 20 הז'אנרים של הסרטים השמורים במאגר של TMDB. כל רשומה שומרת את מזהה הז'אנר ואת השם שלו.
  - **מפתח ראשי**: genre_id – מזהה הז'אנר.
  - **דוגמה לרשומה**:

	genre_id	name
1	28	Action

- **keywords**: טבלת מילות המפתח, המכילה רשומה לכל מילת מפתח שמשויכת ללפחות סרט אחד. כל רשומה שומרת את המזהה של מילת המפתח ואת תוכנה. חשוב לציין שמילת מפתח יכולה בפועל להיות מורכבת מכמה מילים, למשל מילת המפתח עם המזהה 613 היא "new year's eve". כמו כן, לכל סרט יכולות להיות מספר מילות מפתח המשויכות אליו, ועל כך נרחיב בהמשך. דוגמה: לסרט "חופי הברך" עם מרלון ברנדו, משויכות מילות המפתח: murder, suspense, union, dock, longshoremen, pigeon.
  - **מפתח ראשי**: keyword_id – מזהה מילת המפתח.
  - **דוגמה לרשומה**:

	keyword_id	name
1	161944	responsibility

- **people**: טבלת העובדים בתעשיית הסרטים (בימאים/תסריטאים/שחקנים), השומרת רשומה לכל עובד שכוללת את המזהה שלו במערכת, שמו, מינו ולינק לתמונת הפרופיל שלו.
  - **מפתח ראשי**: person_id – מזהה העובד במערכת.
  - **דוגמה לרשומה**:

	person_id	name	gender	profile_path
1	514	Jack Nicholson	2	/hINAkm21g80UbaAxA6rHhOaT5Jk.jpg

טבלאות many-to-many:

- **movie_genres**: כל סרט יכול להיות ממספר ז'אנרים, וייתכנו מספר סרטים מאותו ז'אנר ולכן היחס בין movies לבין genres הוא many-to-many והטבלה הזאת משמשת לקישור ביניהן. כל רשומה בטבלה שומרת מזהה של סרט ומזהה של ז'אנר אחד שלו.
  - **מפתח ראשי**: (tmdb_id, genre_id) – הזוג של מזהה הסרט ומזהה הז'אנר.
  - **מפתח זר**: tmdb_id בטבלה הזאת הוא מפתח זר ל-tmdb_id בטבלת ה-movies, שכן היחס בין הטבלאות הוא many-to-many.
  - **מפתח זר**: genre_id בטבלה הזאת הוא מפתח זר ל-genre_id בטבלת ה-genres, שכן היחס בין הטבלאות הוא many-to-many.
- **movie_keywords**: באמור, לכל סרט יכולות להיות משויכות מספר מילות מפתח וייתכנו מספר סרטים המשויכים לאותה מילת מפתח ולכן היחס בין movies לבין keywords הוא many-to-many והטבלה הזאת משמשת לקישור ביניהן. כל רשומה בטבלה שומרת מזהה של סרט ומזהה של מילת מפתח אחת שלו.
  - **מפתח ראשי**: (tmdb_id, keyword_id) – הזוג של מזהה הסרט ומזהה מילת המפתח.
  - **מפתח זר**: tmdb_id בטבלה הזאת הוא מפתח זר ל-tmdb_id בטבלת ה-movies, שכן היחס בין הטבלאות הוא many-to-many.
  - **מפתח זר**: keyword_id בטבלה הזאת הוא מפתח זר ל-keyword_id בטבלת ה-keywords, שכן היחס בין הטבלאות הוא many-to-many.
- **movie_cast**: בסרטים כמובן יכול להיות יותר משחקן אחד, ושחקנים יכולים לשחק בהרבה סרטים ולכן היחס בין movies לבין people ובפרט לשחקנים מביניהם הוא many-to-many והטבלה הזאת משמשת לקישור ביניהן. כל רשומה בטבלה שומרת את מזהה הסרט, מזהה הקרדיט, מזהה השחקן, מזהה הליהוק, שם הדמות ששיחק ואת הסדר שלו בקרדיטים בסרט (0 = השחקן הראשי של הסרט וכן האלה).
  - **מפתח ראשי**: (tmdb_id, credit_id) השילוב של מזהה הליהוק ומזהה סרט מהווה מפתח בטבלה ולא השילוב של מזהה השחקן ומזהה הסרט שכן ייתכן שאותו שחקן גילם שני תפקידים שונים בסרט, או דיבב יותר מדמות אחת בסרט אנימציה ואז מזהה השחקן יהיה זהה אך מזהה הליהוק שונה משום שמדובר בליהוק לדמות שונה בסרט.
  - **מפתח זר**: tmdb_id בטבלה הזאת הוא מפתח זר ל-tmdb_id בטבלת ה-movies, שכן היחס בין הטבלאות הוא many-to-many.
  - **מפתח זר**: person_id בטבלה הזאת הוא מפתח זר ל-person_id בטבלת ה-people, שכן היחס בין הטבלאות הוא many-to-many.
  - **דוגמה לרשומה**:

	tmdb_id	credit_id	person_id	cast_id	cast_order	character_name
1	164	52fe4221c3a36847f8006399	1932	6	0	Holly Golightly

- **movie_crew**: מהסבר דומה ל-movie_cast, יש יחס many-to-many בין movies לבין people ובפרט לצוות הסרט שאינו השחקנים, והטבלה הזאת משמשת לקשור ביניהן. כל רשומה בטבלה שומרת את מזהה הסרט, מזהה הקרדיט, מזהה העובד, המחלקה שלו (לדוגמה במאי, כותב, מפיק, עורך), ועבודתו (למשל בעבור כותב יכולות להיות מספר אפשרויות).
  - **מפתח ראשי**: (tmdb_id, credit_id) מאותן סיבות כמו בעבור movie_cast.
  - **מפתח זר**: tmdb_id בטבלה הזאת הוא מפתח זר ל-tmdb_id בטבלת ה-movies, שכן היחס בין הטבלאות הוא many-to-many.
  - **מפתח זר**: person_id בטבלה הזאת הוא מפתח זר ל-person_id בטבלת ה-people, שכן היחס בין הטבלאות הוא many-to-many.
  - **דוגמה לרשומה**:

	tmdb_id	credit_id	person_id	department	job
1	77	52fe4214c3a36847f80024cb	525	Directing	Director

טבלאות לקישור נתוני MovieLens ודירוגים:

- **movielens_links**: טבלה המקשרת בין מזהה סרט עפ"י MovieLens ומזהה סרט עפ"י TMDB על מנת שנוכל לבצע join בשאילתות.
  - **מפתח ראשי**: ml_movie_id – מזהה סרט במאגר של MovieLens.
  - **מפתח זר**: tmdb_id בטבלה הזאת הוא מפתח זר ל-tmdb_id בטבלת ה-movies, ע"מ להבטיח מיפוי תקין בין דירוג MovieLens לבין הסרט המדורג.
  - **רשומה לדוגמה**:

	ml_movie_id	imdb_id	tmdb_id
1	1	tt0114709	862

- **ratings**: טבלת דירוגי MovieLens, השומרת רשומה לכל דירוג, הכוללת את מזהה המשתמש שדירג, מזהה הסרט, הדירוג (בסקאלה של 0.5-5) וה-timestamp.
  - **מפתח ראשי**: (user_id, ml_movie_id) – מזהה הדירוג. אותו משתמש לא יכול לדרג את אותו סרט יותר מפעם אחת.
  - **מפתח זר**: ml_movie_id בטבלה הזאת הוא מפתח זר ל-ml_movie_id בטבלת ה-movielens_links, ע"מ להבטיח מיפוי תקין בין דירוג MovieLens לבין הסרט המדורג.
  - **רשומה לדוגמה**:

	user_id	ml_movie_id	rating	ts_unix
1	1	31	2.5	1260759144

Design Reasoning:

בשל מגבלת 100MB בשרת ביצענו עיבוד מקדים לקבצי ה-CSV המקוריים מהמאגר של TMDB, ובמהלכו צמצמנו את הנתונים לכדי נתונים הרלוונטיים ל-8000 סרטים נבחרים. הסרנו רשומות כפולות, רשומות עם מידע חלקי, הפכנו ערכי string ל-float בשדות כמו id ובהתאם לצמצום רשומות טבלת הסרטים, צמצמנו את מספר הרשומות של כל שאר הטבלאות באופן שבו שלא יהיו join-ים ריקים בטבלאות הדירוגים לדוגמה.

קבצי ה-CSV המקוריים מהמאגר של TMDB שמרו attributes כמו genres, cast, crew, keywords במחרוזות בפורמט JSON. אם היינו משאירים זאת כך, היינו מקבלים שחיפוש או סינון ע"י ז'אנר, מילת מפתח או אנשים היה הופך במקרה הטוב לקשה מאוד ולא יעיל ובמקרה הרע לבלתי אפשרי ב-SQL. כמו כן, היינו מאבדים referential integrity שכן אי אפשר היה לאכוף מזהים תקינים. בנוסף שאילתות aggregations כפי שמימשנו היו הופכות ללא יעילות. כלומר השיקול כאן היה בעיקר **יעילות זמן ריצה**.

בנוסף, פיצלנו את טבלת העובדים בתעשיית הסרטים לכדי שחקנים בטבלה movie_cast וכל שאר אנשי הצוות כמו במאים, כותבים ומפיקים בטבלה movie_crew. עשינו זאת משום שיש תכונות הרלוונטיות רק לשחקנים כמו שם הדמות אותה הם מגלמים, ואילו יש תכונות רלוונטיות רק לאנשי צוות כמו המחלקה שלהם והתפקיד שלהם בה. תכנון של ה-database שאינו מבצע את ההפרדה הזאת היה גורר ערכי NULL מרובים בשדות שלא רלוונטיים לעובד.

אז כאמור, אם היינו ממלאים את הטבלאות ישירות מה-CSV המקוריים, היינו מקבלים database שהחיפושים בו לא יעילים והנתונים מעורבבים. העיבוד המקדים כלל שורה של פעולות ניסוי וטעייה כדי לסנן מידע, לנקות אותו, לפצל טבלאות ולהסיר רשומות כדי לעמוד **במגבלות המקום**. החיסרון של כל התהליך הוא בעיקר הפיצול של המידע על פני טבלאות רבות, וכיוצא בזאת, join ייצור טבלאות עצומות שיובילו ל-lookup ופעולות אחרות לעבוד באיטיות. עם זאת, בעזרת אינדקסים הצלחנו לייעל את התהליך כפי שנרחיב מיד.

Indices:

יצרנו 2 אינדקסי full-text שיעזרו לנו בשאילתות:

- **movies**: בטבלה movies יצרנו אינדקס full-text על השדות title (שם הסרט), overview (תקציר הסרט), tagline (סלוגן הסרט, משפט שווקי המופיע לעיתים מתחת לכותרת הסרט בפוסטרים למשל). זאת משום **שבשאילתה 1** רצינו לאפשר למשתמש לחפש **ביעילות** סרטים, ע"י השוואת טקסט חופשי לפרטי הסרט והחזרת תוצאות ממוינות לפי מטריקה הנותנת עדיפות לסרטים המתאימים לטקסט החיפוש.
- **keywords**: בטבלה keywords יצרנו אינדקס full-text על השדה של שם מילת המפתח. זאת משום **שבשאילתה 2** רצינו לאפשר למשתמש לחפש **ביעילות** סרטים עפ"י מילות המפתח המשויכות אליהם.

יצרנו מספר אינדקסים נוספים לייעלו פעולת ה-join בשאילתות השונות, ייעול פעולות חיפוש וכן aggregation:

- **movies**:
  - **PRIMARY KEY (tmdb_id)**: כל טבלה אחרת המבצעת join עם טבלת הסרטים, מבצעת אותו על השדה הזה. לכן הגדרתו כמפתח ראשי מסייעת לביצוע Join ביעילות בשאילתות **2**,**3**,**4**.
- **genres**:
  - **PRIMARY KEY (genre_id)**: מייעל את פעולת ה-join מהטבלה המקשרת בין הז'אנרים לסרטים.
  - **UNIQUE KEY(name)**: מייעלת חיפוש שם של ז'אנר ששחקן לא השתתף בו בשאילתה **5**.
- **movie_genres**:
  - **PRIMARY KEY (tmdb_id, genre_id)**: מונע כפילויות, שכן אותו סרט לא יכול להיות משויך לאותו ז'אנר פעמיים. כמו כן מייעל פעולות Join כאשר אנחנו כבר יודעים את המזהה של הסרט. נעשה שימוש בשאילתות **3**,**5** כאשר רוצים לחפש ז'אנרים של סרט נתון.
- **keywords**:
  - **PRIMARY KEY (keyword_id)**: מונע כפילויות, שכן לא נרצה לשמור פעמיים את אותה מילת מפתח, או שתי מילות מפתח שונות עם אותו מזהה. כמו כן מייעל את פעולת ה-join בין הטבלה ל-movie_keywords.
- **movie_keywords**:
  - **PRIMARY KEY (tmdb_id, keyword_id)**: מונע כפילויות, שכן אי אפשר לשייך סרט לאותה מילת מפתח יותר מפעם אחת. כמו כן מייעל את המעבר מסרט למילות המפתח שלו.
  - **KEY (keyword_id)**: מייעל את המעבר ממילת מפתח מסוימת, לכל הסרטים המשויכים למילת המפתח הזאת. נעשה בכך שימוש בשאילתה **2** המחפשת סרטים לפי מילות מפתח שהתקבלו בקלט.
  - **KEY (keyword_id, tmdb_id)**: מייעל את הפעולה של שאילתה **2** משום שבכך מוחזר tmdb_id מייד ונחסך מאיתנו lookup.
- **people**:
  - **PRIMARY KEY (person_id)**: מייעל את פעולת ה-join כאשר אנחנו צריכים את שם העובד. נעשה בכך שימוש בשאילתות **3** ו-**5** שהן שאילתות על שחקנים, שבסופן נרצה להציג למשתמש את שם השחקן המקיים תכונה מסוימת.
- **movie_cast**:
  - **PRIMARY KEY (tmdb_id, credit_id)**: מזהה ביחידות שחקן בתוך סרט. מונע כפילויות שכן לא ייתכנו שני מזהי קרדיטים זהים בתוך אותו סרט.
  - **KEY (person_id)**: מייעלות חיפוש הסרטים בהם השתתף שחקן מסוים. נעשה בכך שימוש בשאילתות **3**,**5** ששתיהן שאילתות על שחקנים ומונות את הסרטים ששיחקו בהם לצרכים שונים.
  - **KEY (person_id, tmdb_id)**: מייעל את המניה של סרטים ייחודיים לכל שחקן. כמו כן, משפר את ביצועי שאילתות **3**,**5**.
- **movie_crew**:
  - **PRIMARY KEY (tmdb_id, credit_id)**: מזהה ביחידות עובד בתוך סרט. מונע כפילויות שכן לא ייתכנו שני מזהי קרדיטים זהים בתוך אותו סרט.
- **movielens_links**:
  - **PRIMARY KEY (ml_movie_id)**: מייעל את פעולת ה-join מ-ratings.
  - **KEY (tmdb_id)**: מייעלת את שאילתה **4** המתחילה מסרט TMDB ורוצה את דירוג MovieLens שלו דרך ml_movie_id.
- **ratings**:
  - **PRIMARY KEY (user_id, ml_movie_id)**: מוודא שמשתמש לא ידרג את אותו הסרט פעמיים. מייעל את מציאת הדירוגים לסרט מסוים.
  - **KEY (ml_movie_id)**: פעולת ה-GROUP BY בשאילתה **4** הופכת להיות הרבה יותר יעילה כאשר הטבלה מאונדקסת עפ"י ml_movie_id.

Queries:

שאילתה 1: השאילתה מקבלת טקסט ומבצעת חיפוש full-text מעל שדות הטקסט בטבלת הסרטים: השם, הסלוגן והתקציר. היא מחזירה את רשימת הסרטים ממוינת עפ"י מידת הרלוונטיות שחישבה, כאשר שובר השוויון הוא דירוג ה-TMDB של הסרט. מטרתה היא לאפשר חיפוש יעיל של סרטים עפ"י מילים המופיעות בפרטים שלו. החיפוש מתאפשר ביעילות בזכות אינדקס ה-full-text שהגדרנו על שדות הטקסט של טבלת הסרטים.

לדוגמה, אם נרצה למשל למצוא 10 סרטים לכל היותר שהשם/התקציר/הסלוגן שלהם מכיל את המילה "America", נפעיל את הפקודה הבאה:

```
py src\queries_execution.py --q 1 --text "America" --limit 10
```

ונקבל את התוצאה הבאה:

TMDB movie id	Movie title	Release year	Overview	Vote average	Relevance score
100402	Captain America: The Winter Soldier	2014	After the cataclysmic events in New York with The Avengers, Steve Rogers, aka Captain America is living quietly in Washi…	7.6	10.074778
3989	Team America: World Police	2004	Team America World Police follows an international police force dedicated to maintaining global stability. Learning that…	6.6	10.074778
1776	Jesus Camp	2006	A growing number of Evangelical Christians believe there is a revival underway in America that requires Christian youth …	6.6	10.074778
206953	Louis Theroux: America's Most Hated Family in Crisis	2011	Following up on his 2007 documentary, The Most Hated Family in America, Louis Theroux returns to Topeka, Kansas, for a w…	7.5	7.556083
3131	Gangs of New York	2002	It's 1863. America was born in the streets. Amsterdam Vallon returns to the Five Points of America to seek vengeance aga…	7.1	7.556083
3179	Beavis and Butt-Head Do America	1996	Mike Judge's slacker duo, Beavis and Butt-Head, wake to discover their TV has been stolen. Their search for a new one ta…	6.5	7.556083
38225	Wild America	1997	Three brothers - Marshall, Marty and Mark dream of becoming naturalists and portraying animal life of America. One summe…	5.8	7.556083
47249	America America	1963	A young Anatolian Greek, entrusted with his family's fortune, loses it en route to Istanbul and dreams of going to Ameri…	5.8	7.556083
311	Once Upon a Time in America	1984	A former Prohibition-era Jewish gangster returns to the Lower East Side of Manhattan over thirty years later, where he o…	8.3	5.037389
40663	Gasland	2010	It is happening all across America-rural landowners wake up one day to find a lucrative offer from an energy company wan…	7.4	5.037389

שאילתה 2: השאילתה מקבלת טקסט בצורת ביטוי בוליאני ומחפשת את רשימת הסרטים שמילות מפתח המסווגות להם, מתאימות לביטוי. מחזירה למשתמש פרטים על רשימת סרטים זאת, ממוינת לפי מספר מילות המפתח המתאים, וכשובר שוויון מציגה את הסרטים בעלי דירוג TMDB גבוה יותר - ראשונים. מטרת השאילתה היא לאפשר חיפוש יעיל של סרטים באמצעות מילות מפתח המשויכות לכל סרט. החיפוש מתאפשר ביעילות בזכות אינדקס ה-full-text שהגדרנו על שדה הטקסט name של הטבלה keywords.

לדוגמה, אם נרצה למצוא 10 סרטים לכל היותר שמילות המפתח המיוחסות להם מכילות את המילים "love" או "tragedy" או "musical" אבל לא מכילות את המילה "secret", נפעיל את הפקודה הבאה:

```
py src\queries_execution.py --q 2 --text '+(love tragedy musical) -secret' --limit 10
```

ונקבל את התוצאה הבאה:

TMDB movie id	Movie title	Release year	Overview	Vote average	Matched keywords
1725	West Side Story	1961	In the slums of the upper West Side of Manhattan, New York, a gang of Polish-American teenagers called the Jets compete …	7	6
26428	Agora	2009	A historical drama set in Roman Egypt, concerning philosopher Hypatia of Alexandria and her relationship with her slave …	6.9	5
6003	Romeo and Juliet	1968	Director Franco Zeffirelli's beloved version of one of the most well-known love stories in the English language. Romeo M…	7.3	4
5123	August Rush	2007	A drama with fairy tale elements, where an orphaned musical prodigy uses his gift as a clue to finding his birth parents…	7.2	4
2288	Closer	2004	A witty, romantic, and very dangerous love story about chance meetings, instant attractions, and casual betrayals. Two c…	6.7	4
15121	The Sound of Music	1965	Film adaptation of a classic Rodgers and Hammerstein musical based on a nun who becomes a governess for an Austrian fami…	7.4	3
688	The Bridges of Madison County	1995	Photographer Robert Kincaid wanders into the life of housewife Francesca Johnson for four days in the 1960s.	7.3	3
1829	The Story of Adele H	1975	Adèle Hugo, daughter of renowned French writer Victor Hugo, falls in love with British soldier Albert Pinson while livin…	6.9	3
10995	The Lover	1992	A poor French teenage girl engages in an illicit affair with a wealthy Chinese heir in 1920s Saigon. For the first time …	6.5	3
13936	Oklahoma!	1955	This joyous celebration of frontier life combines tender romance and violent passion in the Oklahoma Territory of the 19…	6.3	3

ואכן, לדוגמה לקלאסיקה "סיפור הפרברים" שבראש הרשימה מיוחסות הכי הרבה מילות מפתח רלוונטיות:

based on play or musical | forbidden love | love at first sight | musical | obsessive love | young love

שאילתה 3: השאילתה מקבלת מספר מינימלי של סרטים ששחקן צריך לשחק בהם, על מנת להיכלל ברשימת בשחקנים ששיחקו בכמה שיותר ז'אנרים שונים בקריירה. תפקיד השאילתה היא לאפשר למשתמש למצוא את השחקנים הוורסטיליים ביותר. השאילתה מחזירה את פרטי השחקנים, כאשר ההעדפה הראשונית היא לשחקנים ששיחקו בכמה שיותר ז'אנרים, ובמקרה של שובר שוויון – השחקן ששיחק ביותר סרטים יופיע ראשון.

לדוגמה, אם נרצה למצוא את 10 השחקנים הוורסטיליים ביותר, ששיחקו בלפחות 30 סרטים, נפעיל את הפקודה הבאה:

```
py src\queries_execution.py --q 3 --min-movies 30 --limit 15
```

ונקבל את התוצאה הבאה:

Actor id	Actor name	Profile	Number of distinct genres	Number of movies
2231	Samuel L. Jackson	/AvCReLikjzYEf9XjTQxbv3JWgKT.jpg	18	79
884	Steve Buscemi	/e19GfOWzMNN1hi7B9Ci62hMvtXs.jpg	18	53
85	Johnny Depp	/kbWValANhZI8rbWZXximXuMN4UN.jpg	18	44
1229	Jeff Bridges	/xms1RAY6q7Lzp7wNeRCBOkzhucn.jpg	18	39
10205	Sigourney Weaver	/gxBIAr3CnBjkNRoPovVJCvEGqP0.jpg	18	36
380	Robert De Niro	/lvTSwUcvJRLAJ2FB5qFaukel516.jpg	17	66
192	Morgan Freeman	/oGJQhOpT8S1M56tvSsbEBePV501.jpg	17	56
28633	Richard Jenkins	/iz61iPIgNUp0yk48bRNjEJ5GHO4.jpg	17	51
3896	Liam Neeson	/9mdAohLsDu36WaXV2N3SQ388bvz.jpg	17	50
7447	Alec Baldwin	/zFtp13AZyPbqbj6yHujPWHO5ioh.jpg	17	49
3036	John Cusack	/uKydQYuZ9TnCzvbQLtj6j98vWAT.jpg	17	47
2395	Whoopi Goldberg	/n3lF8w4X4rDa4J2LMDIxMEcuUeH.jpg	17	43
8891	John Travolta	/ns8uZHEHzV18ifqA9secv8c2Ard.jpg	17	43
57755	Woody Harrelson	/1ecdooAHICUhCZKKEKlFtccEmTU.jpg	17	43
2975	Laurence Fishburne	/8suOhUmPbfKqDQ17jQ1Gy0mI3P4.jpg	17	42

ואכן, קיבלנו ש-5 שחקנים שיחקו ב-18/20 ז'אנרים, אך שובר השוויון קובע שסמואל אל ג'קסון יהיה בראש הרשימה בזכות מספר הסרטים שבהם שיחק.

שאילתה 4: השאילתה מקבלת מספר מינימלי של דירוגים שסרט צריך לקבל מ-TMDB ומ-MovieLens כדי להיכלל ברשימת הסרטים עליה נערכת השוואה בין ממוצע הדירוגים בכל פלטפורמה. מחזירה את רשימת הסרטים האלו, ממוינת עפ"י ההפרש בערך מוחלט בין ממוצע דירוג הסרט ב-TMDB לבין ממוצע דירוג בסרט ב-MovieLens. מטרתה היא לאפשר למשתמש למצוא סרטים שנחשבים overrated ביחס לפלטפורמה אחת ו-underrated ביחס לאחרת תוך שמירה על משמעות הדירוגים ככל שמספר הדירוגים המינימלי עולה.

לדוגמה, אם נרצה למצוא את רשימת 15 הסרטים שדורגו ע"י לכל הפחות 100 מדרגים גם ה-TMDB וגם ב-MovieLens וההפרש בין הדירוגים הוא מקסימלי בערך מוחלט, נפעיל את הפקודה הבאה:

```
py src\queries_execution.py --q 4 --min-ratings 100 --limit 15
```

ונקבל את התוצאה הבאה:

TMDB movie id	Movie title	Release year	MovieLens average ratings	MovieLens number of reviews	TMDB Vote average	TMDB number of reviews	Gap
771	Home Alone	1990	5.80534	129	7.1	2487	-1.29466
597	Titanic	1997	6.294038	164	7.5	7770	-1.205962
3049	Ace Ventura: Pet Detective	1994	5.269841	175	6.4	1684	-1.130159
9331	Clear and Present Danger	1994	7.304348	115	6.4	390	0.904348
1934	Shakespeare in Love	1998	7.704316	121	6.8	831	0.904316
9804	Waterworld	1995	5.004916	113	5.9	1017	-0.895084
9598	Babe	1995	6.891892	148	6	756	0.891892
854	The Mask	1994	5.711253	157	6.6	2550	-0.888747
95	Armageddon	1998	5.653884	113	6.5	2540	-0.846116
628	Interview with the Vampire	1994	6.435556	125	7.2	1558	-0.764444
8587	The Lion King	1994	7.283333	200	8	5520	-0.716667
8467	Dumb and Dumber	1994	5.801688	158	6.5	1894	-0.698312
393	Kill Bill: Vol. 2	2004	7.033441	103	7.7	4061	-0.666559
275	Fargo	1996	8.348214	224	7.7	2080	0.648214
114	Pretty Woman	1990	6.356765	147	7	1807	-0.643235

ואכן, לדוגמה הסרט Home Alone דורג ע"י 129 מדרגים ב-MovieLens שקבעו שדירוגו בממוצע הוא 5.8 ו-2487 מדרגים ב-TMDB שקבעו שדירוגו בממוצע הוא 7.1 ולכן הסרט נחשב ל-overrated אם מסכימים עם הצד של MovieLens.

שאילתה 5: השאילתה מקבלת שם של ז'אנר ומספר מינימלי של סרטים ששחקן חייב להשתתף בהם על מנת להיכלל ברשימת השחקנים ששיחקו בהכי הרבה סרטים אך לא שיחקו בז'אנר הזה. מטרת השאילתה להציג למשתמש סטטיסטיקה מעניינת ואולי להצביע על מגמה כמו שחקנים מודרניים שכבר לא משתתפים בז'אנרים שעברו מן העולם, או ז'אנר ששחקנים נוטים להימנע ממנו בקריירה. מחזירה את שם השחקן ואת מספר הסרטים בהם שיחק, כאשר אף אחד מהסרטים האלה הוא לא מהז'אנר שהתקבל כקלט.

לדוגמה, אם נרצה למצוא את 15 השחקנים ששיחקו בלפחות 20 סרטים אך מעולם לא במערבון, נפעיל את הפקודה הבאה:

```
py src\queries_execution.py --q 5 --genre "Western" --min-movies 20 --limit 15
```

ונקבל את התוצאה הבאה:

```
Person id  Name                Profile                          Movies count
-----------------------------------------------------------------------------
380        Robert De Niro      /lvTSwUcvJRLAJ2FB5qFaukel516.jpg  67
62         Bruce Willis        /2B7RySy2WMVJKKEFN2XA3IFb8w0.jpg  62
3895       Michael Caine       /vvj0JMSFpOajXCE46Hy4dyqSP2U.jpg  55
1230       John Goodman        /eOIx8zj1vYIRhVY2bK5cjIQfua0.jpg  53
2963       Nicolas Cage        /ti2h1OS1n1VwoJHWFaJD8dMZuEE.jpg  53
2157       Robin Williams      /sojtJyIV3lkUeThD7A2oHNm8183.jpg  52
4690       Christopher Walken  /ysO1GwRzLT9OVAB9Y2SKHxomqDr.jpg  49
7447       Alec Baldwin        /zFtp13AZyPbqbj6yHujPWHO5ioh.jpg  49
2283       Stanley Tucci       /omGlTJF2IW5r3L3c5y0qkCt3hFr.jpg  48
3036       John Cusack         /uKydQYuZ9TnCzvbQLtj6j98vWAT.jpg  47
5293       Willem Dafoe        /xM5lhOR5tWWdIlFpBDeZJx9opIP.jpg  47
31         Tom Hanks           /pQFoyx7rpO9CJTAb932F2g8Nlho.jpg  46
707        Dan Aykroyd         /h2PT9yZYv5ml5hL9jvCpWBTWgU.jpg   46
1532       Bill Murray         /7BOoOAIA1CnSzFSVSJP7saniQaB.jpg  46
55636      Donald Sutherland   /tPLVaPjxEscGPKS3ieByloa8Mqj.jpg  46
```

ואכן, רוברט דה נירו הוא השחקן ששיחק בהכי הרבה סרטים מבלי לשחק במערבון אחד.

שאילתה 6: (**UPDATE**) שאילתת עדכון המאפשרת למשתמש שזוהה במערכת, לעדכן את דירוג ה-MovieLens שהעניק לסרט. השאילתה מחזירה את רשומת הדירוג בטבלת ה-ratings לאחר העדכון.

- **הערה**: כפי שפירטתי ב-user_manual.pdf, ה-UI יהיה הרבה יותר נגיש למשתמש שכן הוא לא יצטרך להזין את ה-user id שלו ואת ה-id של הסרט לפי MovieLens. השאילתה הזאת תתאפשר רק לאחר שיתחבר, והאתר ישלים את ה-user id שלו לשאילתה בנוסף לרשימת הסרטים שכבר דירג (באמצעות Join פשוט ב-backend).
- **הערה**: הדירוג יכול להיות רק בטווח 0.5 עד 5 ובקפיצות של 0.5 עפ"י הסקאלה של MovieLens. ישנן בדיקות על הערך המתקבל ב-CLI.

לדוגמה, אם משתמש 7 רוצה לעדכן את הדירוג שנתן לסרט 10 לכדי דירוג 4, אז הוא יפעיל:

```
py src\queries_execution.py --q 6 –user-id 7 –ml-movie-id 10 –new-rating 4
```

ויקבל:

```
Updated rows: 1
User id  MovieLens movie id  New rating  Time stamp
-------------------------------------------------
7        10                 4           1767625635
```

- **הערה**: ציינו בחלק ה-indices איך כל שאילתה מיועלת באמצעות האינדקסים.

Code structure & API usage:

יש לנו 4 קבצי קוד:

- **src/create_db_script.py**: משמש ליצירה או לאיפוס של ה-database וכל הטבלאות והאינדקסים.
- **src/api_data_retrtrieve.py**: טוען את קבצי ה-CSV המופיעים בתיקייה data/csvs ומכניס את הערכים המתאימים לרשומות בטבלאות ה-MySQL בסדר הנכון, תוך התחשבות בתלויות המפתחות הזרים בין הטבלאות.
- **src/queries_db_script.py**: מכיל את 5 השאילתות.
- **src/queries_execution.py**: מקשר בין ה-CLI ל-database. מתחבר ל-database, מריץ שאילתה אחת, ומדפיס את התוצאות בצורה קלה לקריאה.

הפרויקט מוגש כאשר ה-database כבר נוצר והרשומות כבר נטענו לטבלאות. באופן כללי יש להריץ את קובץ היצירה פעם אחת, לאחריו את הטעינה ואז את השאילתות לפי הסינטקס המופיע במסמך user_manual.pdf המצורף לפרויקט.